

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №7

«Нелинейные уравнения»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 25

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2025

ПОСТАНОВКА ЗАДАЧИ

Решить нелинейное уравнение трансцендентного вида методами Ньютона, половинного деления и итерационным методом. Подсчитать время исполнения каждого метода.

Данное уравнение:

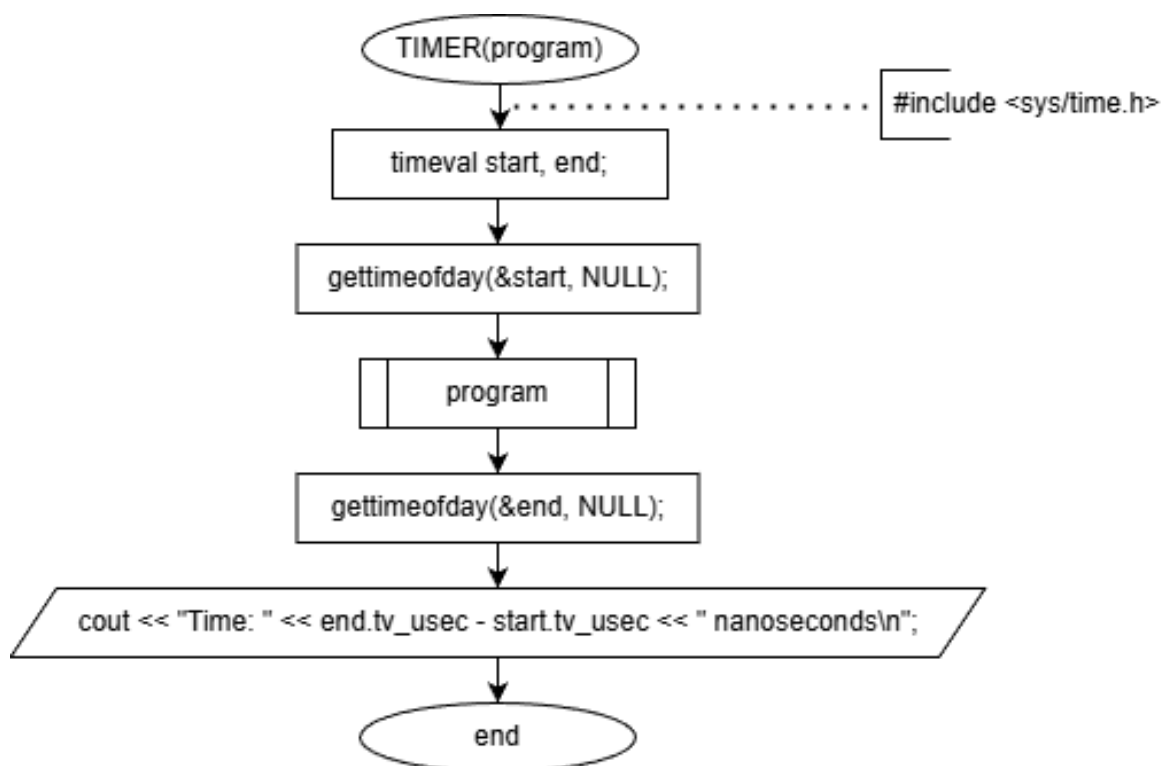
$$x - 2 + \sin \frac{1}{x} = 0$$

Отрезок, содержащий корень: [1; 2].

Эталонное решение: 1,3077.

МЕТОД ПОДСЧЕТА ВРЕМЕНИ ИСПОЛНЕНИЯ

Для удобства подсчета времени исполнения каждого метода был создан макрос `#define TIMER(program)` в файле `timer.h`. Его логика сводится ко взятию системного времени перед выполнением основного алгоритма и после. Затем высчитывается разность этих двух значений и выводится время в наносекундах. Алгоритм макроса можно представить в виде блок-схемы:



Код макроса:

```
#include <sys/time.h>

#define TIMER(program) \
timeval start, end; \
gettimeofday(&start, NULL); \
program \
gettimeofday(&end, NULL);\
cout << "Time: " << end.tv_usec - start.tv_usec << " nanoseconds\n";
```

Помимо подсчета времени, в алгоритмах реализован вывод промежуточных результатов на каждой итерации цикла, что позволяет подсчитать общее число итераций.

АНАЛИЗ РЕШЕНИЯ ПОЛОВИННЫМ ДЕЛЕНИЕМ

1. Начальная точка x_0 выбирается на середине $[a; b] - (a + b) / 2$.

2. Проверяем $[a; x_0]$ и $[x_0; b]$:

2.2. Проверяем значение функции в точках $f(a)$ $f(x_0)$ и $f(b)$: перемножаем $f(a) * f(x_0)$, выбираем интервал с отрицательным знаком.

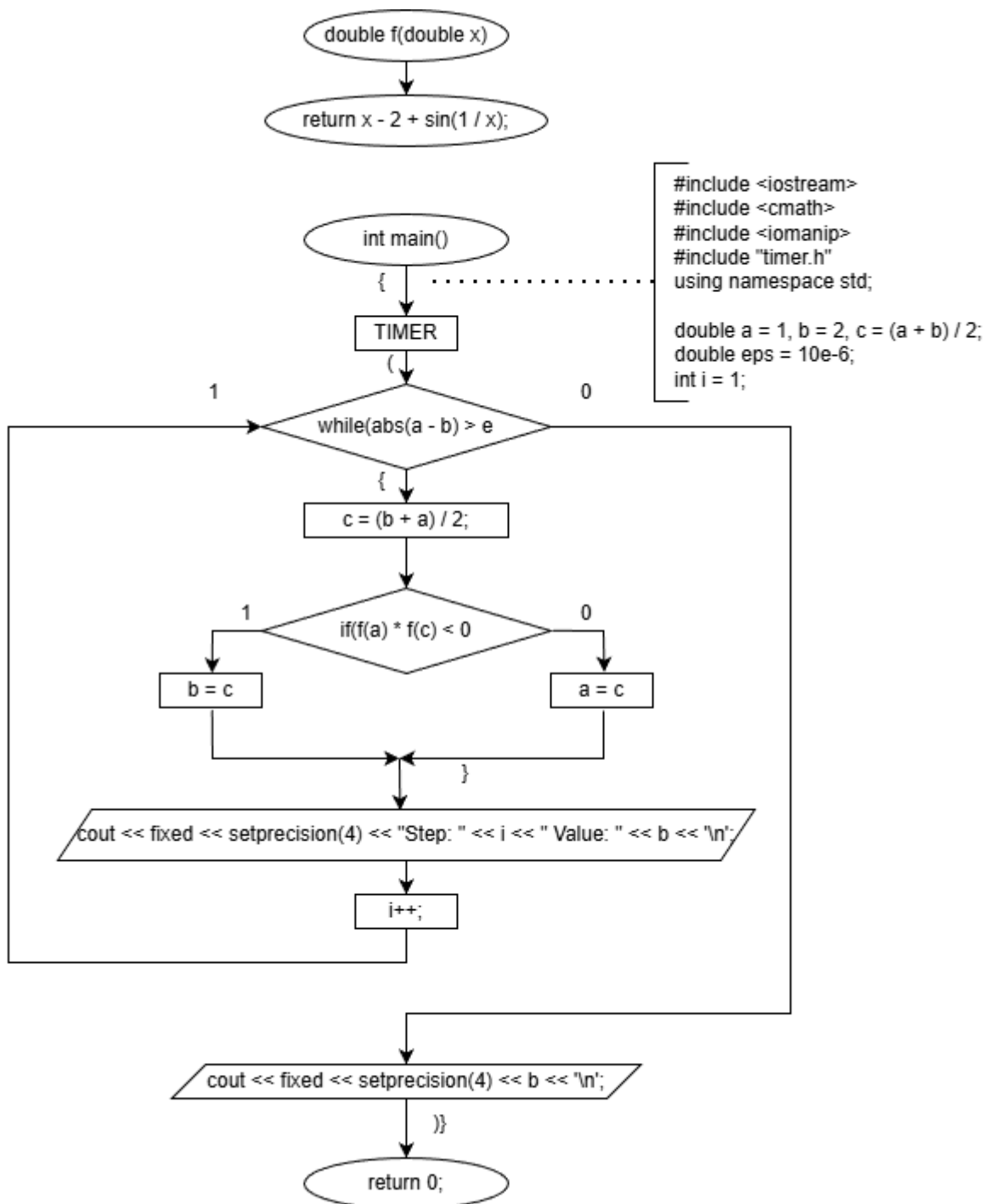
2.4. Переносим границы на новые значения (интервал с отрицательным значением).

2.5. Новая точка x_1 выбирается на середине нового отрезка.

2.6. Имеем два соседних корня x_0 и x_1 .

3. Итерационно находятся значения $x_2, x_3 \dots x_k$, на каждом шагу сравниваются два соседних $|x_i - x_{i+1}|$ с ϵ . То есть, разность между двумя соседними значениями корня будет меньше некоторого заданного ϵ . Точность вычислений $- 10^{-6}$, и при $|x_i - x_{i+1}| \leq \epsilon$ эти корни попадают практически в одну точку, следовательно за искомый корень можно брать любой из них (например, x_{i+1}).

БЛОК-СХЕМА



12/22/2025

КОД

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include "timer.h"
using namespace std;
double f(double x) {
    return x - 2 + sin(1 / x);
}
double a = 1, b = 2, c = (a + b) / 2;
double eps = 10e-6;
int i = 1;
int main() {
    TIMER(
        while (abs(a - b) > eps) {
            c = (b + a) / 2;
            if (f(a) * f(c) < 0) {
                b = c;
            } else {
                a = c;
            }
            cout << fixed << setprecision(4) << "Step: " << i
            << " Value: " << b << '\n';
            i++;
        }
        cout << fixed << setprecision(4) << b << '\n';
    )
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

```
Step: 1 Value: 1.5000
Step: 2 Value: 1.5000
Step: 3 Value: 1.3750
Step: 4 Value: 1.3125
Step: 5 Value: 1.3125
Step: 6 Value: 1.3125
Step: 7 Value: 1.3125
Step: 8 Value: 1.3086
Step: 9 Value: 1.3086
Step: 10 Value: 1.3086
Step: 11 Value: 1.3081
Step: 12 Value: 1.3079
Step: 13 Value: 1.3077
Step: 14 Value: 1.3077
Step: 15 Value: 1.3077
Step: 16 Value: 1.3077
Step: 17 Value: 1.3077
1.3077
Time: 15554 nanoseconds
```

Исходя из выходных данных можно сделать вывод, что метод бисекции выполняется в течении 17 итераций и занимает время, приближенное к 15 миллисекундам.

АНАЛИЗ РЕШЕНИЯ МЕТОДОМ ПРОСТЫХ ИТЕРАЦИЙ

1. Производится релаксация:

- 1.1. Находятся значения производной от данной функции в точках a, b :

$$f'(x) = 1 - \frac{\cos\left(\frac{1}{x}\right)}{x^2}$$

- 1.2. Находятся экстремумы первой производной.

$$f''(x) = \frac{2 \cos\left(\frac{1}{x}\right) x - \sin\left(\frac{1}{x}\right)}{x^4}$$

Вторая производная принимает нулевые значения вне заданного диапазона поиска. Таким образом, минимальное и максимальное значение производной достигается в точках a, b .

- 1.3. По формуле

$$\lambda = \frac{2}{m + M}$$

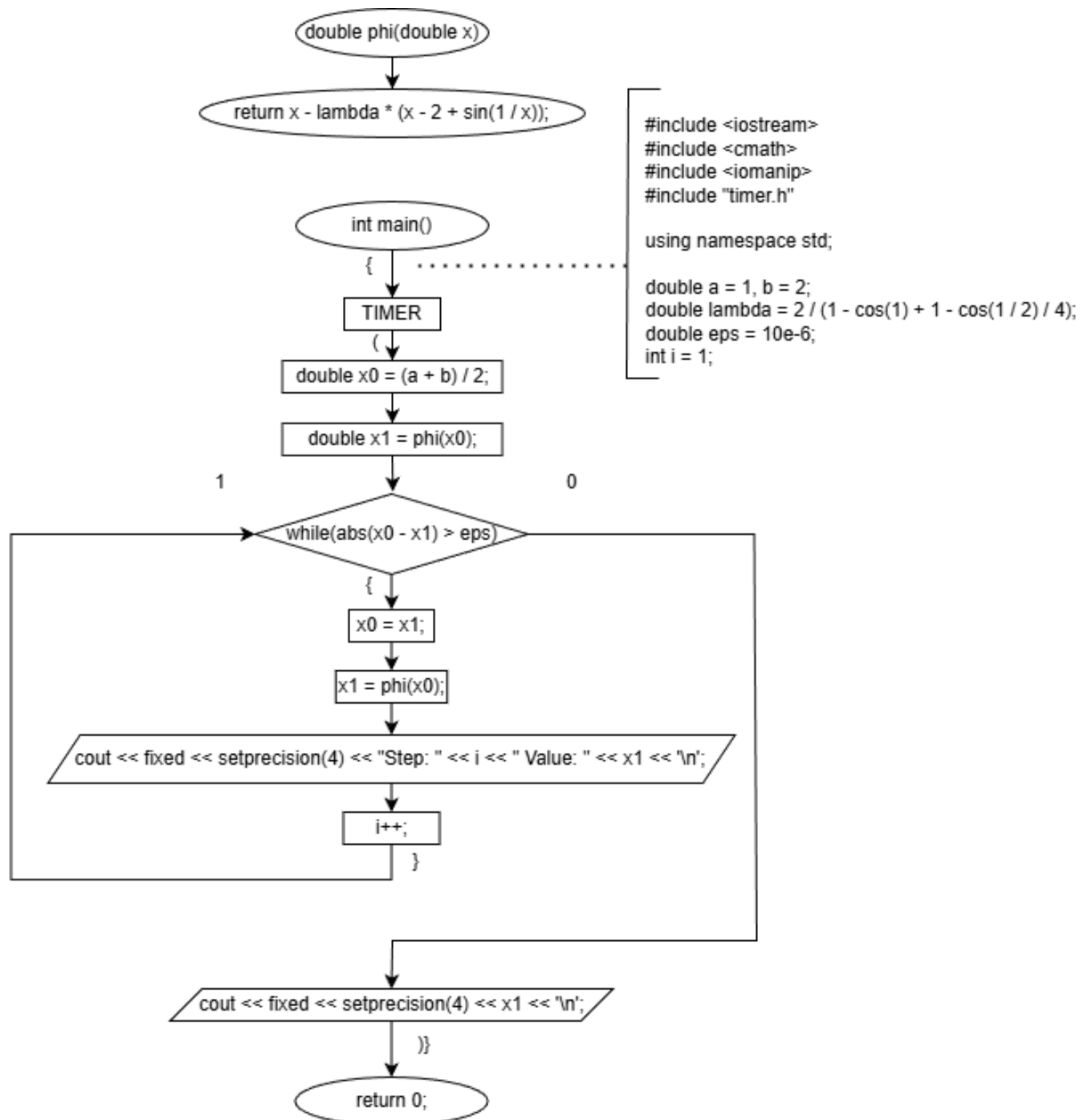
Где m и M – минимальное и максимальное значение производной на отрезке соответственно, находим множитель релаксации.

- 1.4. Итоговое уравнение для $\varphi(x)$ будет выглядеть следующим образом:

$$\varphi(x) = x - \lambda * (x - 2 + \sin\left(\frac{1}{x}\right))$$

2. Для более быстрой сходимости выберем приближение $x_0 = (a + b) / 2$.
3. Каждое последующее приближение высчитывается по формуле $x_{i+1} = \varphi(x_i)$
4. Итерационно находятся значения $x_2, x_3 \dots x_k$, на каждом шагу сравниваются два соседних $|x_i - x_{i+1}|$ с ε . То есть, разность между двумя соседними значениями корня будет меньше некоторого заданного ε . Точность вычислений – 10^{-6} , и при $|x_i - x_{i+1}| \leq \varepsilon$ эти корни попадают практически в одну точку, следовательно за искомым корень можно брать любой из них (например, x_{i+1}).

БЛОК-СХЕМА



КОД

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include "timer.h"

using namespace std;

double a = 1, b = 2;
double lambda = 2 / (1 - cos(1) + 1 - cos(1 / 2) / 4);
double eps = 10e-6;
int i = 1;

double phi(double x) {
    return x - lambda * (x - 2 + sin(1 / x));
}

int main() {
    TIMER(
        double x0 = (a + b) / 2;
        double x1 = phi(x0);

        while (abs(x0 - x1) > eps) {
            x0 = x1;
            x1 = phi(x0);
            cout << fixed << setprecision(4) << "Step: " << i
                << " Value: " << x1 << '\n';
            i++;
        }

        cout << fixed << setprecision(4) << x1 << '\n';
    )
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

```
Step: 1 Value: 1.3075
Step: 2 Value: 1.3077
Step: 3 Value: 1.3077
1.3077
Time: 3717 nanoseconds
```

Исходя из выходных данных можно сделать вывод, что итерационный метод выполняется в течении 3 итераций и занимает время, приближенное к 4 миллисекундам.

АНАЛИЗ РЕШЕНИЯ МЕТОДОМ НЬЮТОНА

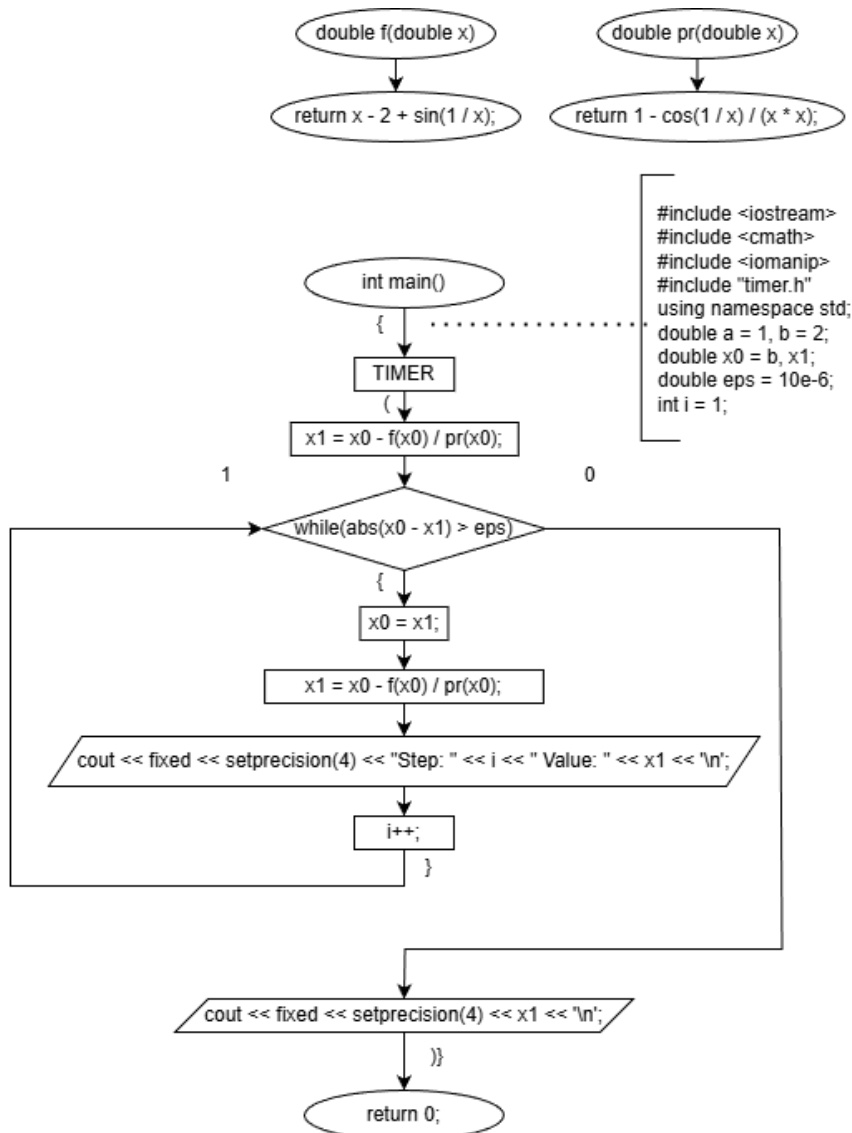
1. Так как $f(b) * f''(b) > 0$, то начальное приближение выбирается справа: $x_0 = b$.

2. Каждое последующее приближение вычисляется по формуле

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

3. Итерационно находятся значения $x_2, x_3 \dots x_k$, на каждом шагу сравниваются два соседних $|x_i - x_{i+1}|$ с ϵ . То есть, разность между двумя соседними значениями корня будет меньше некоторого заданного ϵ . Точность вычислений – 10^{-6} , и при $|x_i - x_{i+1}| \leq \epsilon$ эти корни попадают практически в одну точку, следовательно за искомым корень можно брать любой из них (например, x_{i+1}).

БЛОК-СХЕМА



КОД

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include "timer.h"

using namespace std;

double a = 1, b = 2;
double x0 = b, x1;
double eps = 10e-6;
int i = 1;

double f(double x) {
    return x - 2 + sin(1 / x);
}

double pr(double x) {
    return 1 - cos(1 / x) / (x * x);
}

int main() {
    TIMER(
        x1 = x0 - f(x0) / pr(x0);
        while (abs(x0 - x1) > eps) {
            x0 = x1;
            x1 = x0 - f(x0) / pr(x0);
            cout << fixed << setprecision(4) << "Step: " << i << " Value: "
<< x1 << '\n';
            i++;
        }

        cout << fixed << setprecision(4) << x1 << '\n';
    )
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

```
Step: 1 Value: 1.3096
Step: 2 Value: 1.3077
Step: 3 Value: 1.3077
1.3077
Time: 2969 nanoseconds
```

Исходя из выходных данных можно сделать вывод, что метод Ньютона выполняется в течении 3 итераций и занимает время, приближенное к 3 миллисекундам.

ВЫВОД

В ходе выполнения работы было решено нелинейное трансцендентное уравнение тремя методами: методом половинного деления, методом простых итераций и методом Ньютона. Для каждого метода была достигнута заданная точность и было получено эталонное решение.

Также было произведено сравнение производительности методов:

Название метода	Время (мс)	Количество итераций
Метод половинного деления	16	17
Метод простых итераций	4	3
Метод Ньютона	3	3

При заданном $\varepsilon = 10^{-6}$ метод половинного деления показал самый медленный результат, а методы простых итераций и Ньютона – практически одинаковые. Однако при использовании меньшего эпсилон, например, $\varepsilon = 10^{-15}$, разница между последними двумя методами окажется выше.

Название метода	Время (мс)	Количество итераций
Метод половинного деления	43	47
Метод простых итераций	9	10
Метод Ньютона	4	5

Таким образом можно сделать вывод, что самым быстрым является метод Ньютона, за ним идет метод простых итераций, а самым медленным является метод бисекции.